Problem Statement 1

Online Order Processing System

Business Scenario

You are building the backend service for an **online food ordering application**.

The system should accept customer orders, store them in a database, process orders asynchronously, simulate payment, cooking, and delivery delays, and update the order status until completion.

Tech Stack

- Node.js
- Express.js
- MongoDB
- JavaScript

Asynchronous programming must be implemented using:

- Callback
- Promise
- async / await
- Event Loop behavior

Functional Requirements

1. API Endpoints

Method	Endpoint	Description
POST	/orders	Create a new order
GET	/orders/:id	Retrieve order status
GET	/orders	Retrieve all orders

2. MongoDB Schema

Order Collection

```
{
  "customerName": "Kanna",
  "items": ["Burger", "Pizza"],
  "totalAmount": 450,
  "status": "PLACED",
```

"createdAt": "timestamp"

}

Order Status Flow

PLACED → PAID → COOKING → OUT_FOR_DELIVERY → DELIVERED

Asynchronous Processing Requirements

3. Payment Processing (Callback-Based)

- Payment processing must be implemented using a callback.
- Payment must take 3 seconds.
- If the total amount exceeds 1000, payment must fail.

4. Payment Refactor (Promise-Based)

- The payment logic must be refactored using Promise.
- Promise resolution must indicate successful payment.
- Promise rejection must indicate payment failure.

5. End-to-End Order Workflow (async / await)

The complete order lifecycle must follow this sequence:

1. Save order to MongoDB
2. Process payment
3. Start cooking
4. Assign delivery
5. Update order status after each step

Each step must be asynchronous and implemented using async / await.

Event Loop Demonstration

The following logs must be executed inside the order creation API:

```
console.log("Order received");

setTimeout(() => console.log("Timer completed"), 0);

Promise.resolve().then(() => console.log("Promise resolved"));

console.log("Order processing started");
```

Error Handling

Scenario	Behavior
Payment failure	Order status updated to FAILED
Invalid order ID	HTTP 404
Database error	HTTP 500

All errors must be handled using try / catch and appropriate HTTP status codes.

Problem Statement 2

Online Movie Ticket Booking System

Business Scenario

You are building the backend service for an **online movie ticket booking application**.

The system should allow users to view available movies, book tickets, process payments, manage seat availability, and generate booking confirmations.

Tech Stack

- Node.js
- Express.js
- MongoDB
- JavaScript

Asynchronous programming must be implemented using:

- Callback
- Promise
- async / await
- Event Loop behavior

Functional Requirements

1. API Endpoints

Method	Endpoint	Description
POST	/bookings	Book movie tickets

Method	Endpoint	Description
GET	/bookings/:id	Retrieve booking details
GET	/movies	Retrieve available movies

2. MongoDB Schema

Movie Collection

```
{
 "movieName": "Interstellar",
 "totalSeats": 100,
 "availableSeats": 75
}
```

Booking Collection

```
{
 "userName": "Kanna",
 "movieId": "ObjectId",
 "seatsBooked": 2,
 "status": "BOOKED",
 "createdAt": "timestamp"
}
```

Asynchronous Processing Requirements

---

3. Seat Availability Check (Callback-Based)

- Seat availability must be checked using a callback.
- A delay must be simulated using setTimeout.
- If requested seats exceed available seats, the booking must fail.

4. Payment Processing (Promise-Based)

- Payment processing must be implemented using Promise.
- Payment must take 3 seconds.
- If more than 6 seats are booked, payment must fail.

5. End-to-End Booking Workflow (async / await)

The complete booking lifecycle must follow this sequence:

1. Fetch movie details
2. Check seat availability
3. Process payment
4. Update available seat count
5. Save booking details

All steps must be asynchronous and implemented using async / await.

Event Loop Demonstration

The following logs must be executed inside the booking API:

```
console.log("Booking request received");

setTimeout(() => console.log("setTimeout executed"), 0);

Promise.resolve().then(() => console.log("Promise executed"));

console.log("Booking processing started");
```

Error Handling

Scenario	Behavior
No seats available	HTTP 400
Payment failure	Booking status updated to FAILED
Movie not found	HTTP 404
Database error	HTTP 500

All errors must be handled using try / catch and proper HTTP status codes.

Problem Statement 3

Online Customer Support Ticket Management System

Business Scenario

You are building the backend service for an **online customer support ticket management system** similar to enterprise support platforms.

The system should allow customers to raise support tickets, process tickets asynchronously, assign tickets to support agents, and track ticket status until resolution.

Tech Stack

- Node.js
- Express.js
- MongoDB
- JavaScript

Asynchronous programming must be used using:

- Callback
- Promise
- async / await
- Event Loop behavior

Functional Requirements

1. API Endpoints

Method	Endpoint	Description
POST	/tickets	Create a new support ticket
GET	/tickets/:id	Retrieve ticket status
GET	/tickets	Retrieve all tickets

2. MongoDB Schema

Ticket Collection

```
{
  "customerName": "Kanna",
  "issueType": "PAYMENT",
```

"description": "Payment failed but amount deducted",

"priority": "HIGH",

"status": "CREATED",

"createdAt": "timestamp"

}

Ticket Status Flow

CREATED → ANALYZING → ASSIGNED → IN_PROGRESS → RESOLVED

Asynchronous Processing Requirements

3. Ticket Analysis (Callback-Based)

- Ticket analysis must be implemented using a callback.
- Analysis should be delayed using setTimeout (2 seconds).
- If the description length is less than 10 characters, the ticket must be marked as INVALID.
- Valid tickets must move to the ANALYZING status.

4. Agent Assignment (Promise-Based)

- Ticket assignment must be implemented using a Promise.
- Assignment must take 3 seconds.
- HIGH priority tickets must be assigned immediately.
- LOW priority tickets must be assigned with delay or rejection.

5. End-to-End Workflow (async / await)

The complete ticket lifecycle must follow this sequence:

1. Save ticket details to MongoDB
2. Analyze the ticket
3. Assign a support agent
4. Update ticket status to IN_PROGRESS
5. Resolve the ticket after a delay

All database operations must use async / await.
Ticket status must be updated after each stage.

Event Loop Demonstration

The following logs must be executed inside the ticket creation API:

```
console.log("Ticket received");

setTimeout(() => console.log("setTimeout executed"), 0);

Promise.resolve().then(() => console.log("Promise executed"));

console.log("Ticket processing started");
```

Error Handling

Scenario	Behavior
Invalid ticket	Status updated to REJECTED
Ticket not found	HTTP 404
Assignment failure	Status updated to FAILED
Database error	HTTP 500

All errors must be handled using try / catch and proper HTTP status codes.